

Compiler Design Practical Lab Manual

Complete 10 Practical Experiments with C, Lex/Yacc, GCC Internals & Viva-Voce Bank

Author: Shaswat Raj (BIT Mesra CSE) Academic Scheme: NEP 2024–25 (5th Sem) Credits: 1.5 Practical Credits

Experiment 1: GCC & LLVM Compiler Internals & Optimization Benchmarking

Objective: Study the multi-pass architecture of industrial compilers (GCC/LLVM), inspect intermediate representations (`GIMPLE`, `RTL`), and benchmark runtime execution performance between non-optimized (`-O0`) and highly optimized (`-O3`, `-Ofast`) machine code binaries.

Theoretical Foundations

Modern compilers decouple frontends (Clang/GCC Frontends) from target backends using SSA-based Intermediate Representations (IR). Optimization flags trigger global passes: Loop unrolling, vectorization (SIMD AVX-512), function inlining, and dead-code elimination.

```
# 1. Inspect GCC compilation passes and dump internal representations
gcc -fdump-tree-all -fdump-rtl-all -S matrix_bench.c

# 2. Compile without optimization (-O0) and with aggressive vectorization (-O3)
gcc -O0 -o bench_00 matrix_bench.c
gcc -O3 -march=native -o bench_03 matrix_bench.c

# 3. Benchmark runtime performance with high-precision time command
perf stat ./bench_00
perf stat ./bench_03
```



Complete C Source Code: Matrix Multiplication Benchmark (`matrix\_bench.c`)

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define N 1024
double A[N][N], B[N][N], C[N][N];

void multiply() {
    for (int i = 0; i < N; i++) {
        for (int k = 0; k < N; k++) {
            for (int j = 0; j < N; j++) {
                C[i][j] += A[i][k] * B[k][j];
            }
        }
    }
}

int main() {
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++) {
            A[i][j] = (double)(i + j) / N;
            B[i][j] = (double)(i - j) / N;
        }

    struct timespec start, end;
    clock_gettime(CLOCK_MONOTONIC, &start);
    multiply();
    clock_gettime(CLOCK_MONOTONIC, &end);

    double elapsed = (end.tv_sec - start.tv_sec) + (end.tv_nsec - start.tv_nsec) / 1e9;
    printf("Matrix (%dx%d) Multiply Time: %.4f seconds\n", N, N, elapsed);
    return 0;
}
```

Observation & Analysis: Loop interchange (`i-k-j` vs `i-j-k`) and `-O3` auto-vectorization slashes execution time by > 85% due to continuous spatial cache locality!

Experiment 2: Handcrafted Lexical Analyzer in Pure C (State Machine Paradigm)

Objective: Design and implement a deterministic finite automaton (DFA) based lexical analyzer in pure C without automated generator tools to tokenize C source code into Keywords, Identifiers, Integer/Float Literals, Relational/Arithmetic Operators, and strip multiline comments.

```

#include <stdio.h>
#include <ctype.h>
#include <string.h>
#include <stdlib.h>

const char *keywords[] = {"int", "float", "if", "else", "while", "return", "void", "char", "for"};
int is_keyword(const char *str) {
    for (int i = 0; i < 9; i++) {
        if (strcmp(keywords[i], str) == 0) return 1;
    }
    return 0;
}

void analyze(FILE *fp) {
    int c;
    char buffer[256];
    int idx = 0;

    while ((c = fgetc(fp)) != EOF) {
        if (isspace(c)) continue;

        // Strip single line comments
        if (c == '/') {
            int next = fgetc(fp);
            if (next == '/') {
                while ((c = fgetc(fp)) != EOF && c != '\n');
                continue;
            } else if (next == '*') {
                while ((c = fgetc(fp)) != EOF) {
                    if (c == '*' && (next = fgetc(fp)) == '/') break;
                }
                continue;
            } else {
                ungetc(next, fp);
            }
        }

        // Identifiers or Keywords
        if (isalpha(c) || c == '_') {
            idx = 0;
            buffer[idx++] = c;
            while ((c = fgetc(fp)) != EOF && (isalnum(c) || c == '_')) {
                buffer[idx++] = c;
            }
            buffer[idx] = '\0';
            ungetc(c, fp);
            if (is_keyword(buffer))
                printf("<KEYWORD, %s>\n", buffer);
            else
                printf("<IDENTIFIER, %s>\n", buffer);
        }

        // Numeric Constants (Integer & Floating Point)
        else if (isdigit(c)) {
            idx = 0;
            buffer[idx++] = c;
            int is_float = 0;
            while ((c = fgetc(fp)) != EOF && (isdigit(c) || c == '.')) {
                if (c == '.') is_float = 1;
                buffer[idx++] = c;
            }
            buffer[idx] = '\0';
            ungetc(c, fp);
            printf("<NUMERIC_%s, %s>\n", is_float ? "FLOAT" : "INT", buffer);
        }

        // Operators & Delimiters
        else if (strchr("+-*/%=>!", c)) {
            char op[3] = {c, '\0', '\0'};
            int next = fgetc(fp);
            if ((c == '=' || c == '!' || c == '<' || c == '>') && next == '=') {
                op[1] = next;
            } else {
                ungetc(next, fp);
            }
            printf("<OPERATOR, %s>\n", op);
        }

        else if (strchr("();{}[]", c)) {
            printf("<DELIMITER, %c>\n", c);
        }
    }
}

```

```
}  
}
```

Experiment 3: Lexical Analyzer Specification using Flex / Lex Tool

Objective: Construct an industry-grade lexical analyzer using Flex regular definitions to count words, lines, characters, recognize identifiers, integer/hex constants, and generate a structured token stream.

```

%{
#include <stdio.h>
int line_count = 1;
int token_count = 0;
%}

DIGIT      [0-9]
HEX_DIGIT  [0-9a-fA-F]
LETTER     [a-zA-Z_]
ID         {LETTER}{LETTER}|{DIGIT})*
INT_CONST  {DIGIT}+
HEX_CONST  0[xX]{HEX_DIGIT}+
FLOAT_CONST {DIGIT}+\. {DIGIT}+([eE][+-]?{DIGIT}+)?

%%
\n          { line_count++; }
[ \t\r]+    { /* Ignore whitespace */ }
"//".*      { /* Ignore single-line comment */ }
"/*(^*)|\\*+[^*/])*\\" { /* Ignore multi-line comment */ }

"if"|"else"|"while"|"for"|"int"|"float"|"return"|"void" {
    token_count++;
    printf("Line %d: [KEYWORD]      %s\n", line_count, yytext);
}

{HEX_CONST} {
    token_count++;
    printf("Line %d: [HEX_CONST]   %s (Value = %ld)\n", line_count, yytext, strtol(yytext, NULL, 16));
}

{FLOAT_CONST} {
    token_count++;
    printf("Line %d: [FLOAT_CONST] %s\n", line_count, yytext);
}

{INT_CONST} {
    token_count++;
    printf("Line %d: [INT_CONST]    %s\n", line_count, yytext);
}

{ID} {
    token_count++;
    printf("Line %d: [IDENTIFIER]   %s\n", line_count, yytext);
}

"="|"!="|"<="|">="|"&"|"||"|"+"|"-"|"*"|" "/"|"=" {
    token_count++;
    printf("Line %d: [OPERATOR]     %s\n", line_count, yytext);
}

";"|"|"|"(")"|"{"}"|"["]" {
    token_count++;
    printf("Line %d: [DELIMITER]    %s\n", line_count, yytext);
}

. { printf("Line %d: [LEX_ERROR]   Unrecognized character '%s'\n", line_count, yytext); }
%%

int yywrap() { return 1; }

int main(int argc, char **argv) {
    if (argc > 1) {
        FILE *file = fopen(argv[1], "r");
        if (!file) { perror("fopen"); return 1; }
        yyin = file;
    }
    yylex();
    printf("\nTotal Lines: %d | Total Tokens Scanned: %d\n", line_count, token_count);
    return 0;
}

```

Experiment 4: Recursive Descent Predictive Parser in C

Objective: Implement a top-down Recursive Descent Parser with backtrack-free single-token lookahead for the arithmetic expression grammar with left-recursion eliminated:

$$E \rightarrow TE', \quad E' \rightarrow +TE' \mid \epsilon, \quad T \rightarrow FT', \quad T' \rightarrow *FT' \mid \epsilon, \quad F \rightarrow (E) \mid id$$

```
#include <stdio.h>
#include <ctype.h>
#include <stdlib.h>

char *cursor;
void E(); void E_prime(); void T(); void T_prime(); void F();
void error() { printf("\n❌ Syntax Error: Unexpected token '%c'\n", *cursor); exit(1); }

void match(char expected) {
    if (*cursor == expected) {
        printf("Matched '%c'\n", expected);
        cursor++;
    } else error();
}

void E() {
    printf("E → T E'\n");
    T();
    E_prime();
}

void E_prime() {
    if (*cursor == '+') {
        printf("E' → + T E'\n");
        match('+');
        T();
        E_prime();
    } else {
        printf("E' → epsilon\n");
    }
}

void T() {
    printf("T → F T'\n");
    F();
    T_prime();
}

void T_prime() {
    if (*cursor == '*') {
        printf("T' → * F T'\n");
        match('*');
        F();
        T_prime();
    } else {
        printf("T' → epsilon\n");
    }
}

void F() {
    if (*cursor == '(') {
        printf("F → ( E )\n");
        match('(');
        E();
        match(')');
    } else if (isalnum(*cursor)) {
        printf("F → id (%c)\n", *cursor);
        cursor++;
    } else error();
}

int main() {
    char input[100];
    printf("Enter arithmetic expression (e.g., a+b*c): ");
    scanf("%s", input);
    cursor = input;
    E();
    if (*cursor == '\0')
        printf("\n✅ Parsing Successful! Input is grammatically valid.\n");
    else
        error();
    return 0;
}
```

Experiment 5: Computation of FIRST & FOLLOW Sets in C

Objective: Write a C program to parse Context-Free Grammar productions, recursively compute $\text{FIRST}(A)$ and $\text{FOLLOW}(A)$ sets for all non-terminals, and check if the grammar satisfies the LL(1) condition.

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>

int n; // Number of productions
char prod[20][20];
char first_set[10][20], follow_set[10][20];

void find_first(char c, char *result) {
    if (!isupper(c)) { // Terminal
        if (!strchr(result, c)) {
            int len = strlen(result);
            result[len] = c;
            result[len+1] = '\0';
        }
        return;
    }
    for (int i = 0; i < n; i++) {
        if (prod[i][0] == c) {
            if (prod[i][2] == '$') { // Epsilon
                if (!strchr(result, '$')) {
                    int len = strlen(result);
                    result[len] = '$';
                    result[len+1] = '\0';
                }
            } else {
                for (int j = 2; prod[i][j] != '\0'; j++) {
                    char sub_res[20] = "";
                    find_first(prod[i][j], sub_res);
                    for (int k = 0; sub_res[k] != '\0'; k++) {
                        if (sub_res[k] != '$' && !strchr(result, sub_res[k])) {
                            int len = strlen(result);
                            result[len] = sub_res[k];
                            result[len+1] = '\0';
                        }
                    }
                }
                if (!strchr(sub_res, '$')) break;
            }
        }
    }
}

int main() {
    // ... (main function body) ...
}
```

Experiment 6: Shift-Reduce Parser Simulation with Explicit Stack

Objective: Implement a bottom-up Shift-Reduce Parser in C that maintains an explicit parser stack and input buffer, displaying the complete step-by-step Shift, Reduce, and Accept actions.

Shift-Reduce Parser Execution Trace Matrix for `id + id \* id`

Step	Parser Stack	Input Buffer	Action Taken
1	`\$`	`id + id * id \$`	**Shift** `id`
2	`\$ id`	`+ id * id \$`	**Reduce** $F \rightarrow id$
3	`\$ F`	`+ id * id \$`	**Reduce** $T \rightarrow F$
4	`\$ T`	`+ id * id \$`	**Reduce** $E \rightarrow T$
5	`\$ E`	`+ id * id \$`	**Shift** `+`
6	`\$ E +`	`id * id \$`	**Shift** `id`
7	`\$ E + id`	`* id \$`	**Reduce** $F \rightarrow id$
8	`\$ E + F`	`* id \$`	**Reduce** $T \rightarrow F$
9	`\$ E + T`	`* id \$`	**Shift** `*`
10	`\$ E + T *`	`id \$`	**Shift** `id`
11	`\$ E + T * id`	`\$`	**Reduce** $F \rightarrow id$
12	`\$ E + T * F`	`\$`	**Reduce** $T \rightarrow T * F$
13	`\$ E + T`	`\$`	**Reduce** $E \rightarrow E + T$
14	`\$ E`	`\$`	**ACCEPT (Valid Syntax!)**

Experiment 7: Syntax-Directed Calculator using Lex & Yacc (Bison)

Objective: Develop an interactive mathematical expression calculator with operator precedence, associativity declarations (`%left`, `%right`), variable memory storage, and syntax tree evaluation using Lex and Yacc.


```

/* --- calc.l (Lex Specification) --- */
%{
#include "calc.tab.h"
#include <stdlib.h>
%}

%%
[0-9]+      { yylval.val = atoi(yytext); return NUMBER; }
[a-zA-Z]    { yylval.var = yytext[0]; return VARIABLE; }
[ \t]       ; /* Ignore spaces */
\n          { return '\n'; }
"+"         { return '+'; }
"-"         { return '-'; }
"*"         { return '*'; }
"/"         { return '/'; }
"="         { return '='; }
"("         { return '('; }
")"         { return ')'; }
.           { yyerror("Unknown character"); }
%%

int yywrap() { return 1; }

/* --- calc.y (Yacc / Bison Specification) --- */
%{
#include <stdio.h>
#include <stdlib.h>
int sym[26];
void yyerror(const char *s) { fprintf(stderr, "Parse Error: %s\n", s); }
int yylex();
%}

%union { int val; char var; }
%token <val> NUMBER
%token <var> VARIABLE
%type <val> expr

%left '+' '-'
%left '*' '/'
%right UMINUS

%%
program:
    program statement '\n'
    | /* empty */
    ;

statement:
    expr                                { printf("Result = %d\n", $1); }
    | VARIABLE '=' expr                 { sym[$1 - 'a'] = $3; printf("%c = %d\n", $1, $3); }
    ;

expr:
    NUMBER                             { $$ = $1; }
    | VARIABLE                         { $$ = sym[$1 - 'a']; }
    | expr '+' expr                     { $$ = $1 + $3; }
    | expr '-' expr                     { $$ = $1 - $3; }
    | expr '*' expr                     { $$ = $1 * $3; }
    | expr '/' expr                     {
        if ($3 == 0) { yyerror("Division by zero!"); $$ = 0; }
        else $$ = $1 / $3;
    }
    | '-' expr %prec UMINUS             { $$ = -$2; }
    | '(' expr ')'                     { $$ = $2; }
    ;
%%

int main() {
    printf("\n🧮 Interactive Lex/Yacc Calculator Ready (Enter expressions):\n");
    yyparse();
    return 0;
}

```

Experiment 8: Intermediate Three-Address Code (TAC) Generation

Objective: Generate Three-Address Code (TAC) in Quadruple representation for assignment statements and boolean expressions using Yacc action attributes.

```

#include <stdio.h>
#include <string.h>

int temp_count = 1;
char* new_temp() {
    static char temp_name[10];
    sprintf(temp_name, "t%d", temp_count++);
    return temp_name;
}

void emit_quad(const char *op, const char *arg1, const char *arg2, const char *res) {
    printf("%-8s %-8s %-8s %-8s\n", op, arg1, arg2, res);
}

int main() {
    printf("Generated Quadruples for 'a = b * -c + b * -c':\n");
    printf("%-8s %-8s %-8s %-8s\n", "OP", "ARG1", "ARG2", "RESULT");
    printf("-----\n");
    emit_quad("uminus", "c", "_", "t1");
    emit_quad("*", "b", "t1", "t2");
    emit_quad("uminus", "c", "_", "t3");
    emit_quad("*", "b", "t3", "t4");
    emit_quad("+", "t2", "t4", "t5");
    emit_quad("=", "t5", "_", "a");
    return 0;
}

```

Experiment 9: Control Flow & Boolean Backpatching Simulation

Objective: Implement Backpatching algorithms (`makelist`, `merge`, `backpatch`) for `if-else` and `while` loop intermediate code generation without requiring a second compiler pass.

Backpatching Algorithm Trace for `if (a < b) x = 1; else x = 2;`

1. `100: if a < b goto \_\_\_` (True list = {100})
2. `101: goto \_\_\_` (False list = {101})
3. `102: x = 1` $\implies$ `backpatch(True\_list, 102)`
4. `103: goto \_\_\_` (Next list = {103})
5. `104: x = 2` $\implies$ `backpatch(False\_list, 104)`
6. `105: ...` $\implies$ `backpatch(Next\_list, 105)`

Experiment 10: Machine-Independent Code Optimization in C

Objective: Implement local optimizations on a basic block representation in C: Constant Folding, Constant Propagation, and Dead Code Elimination.

```
#include <stdio.h>
#include <ctype.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    char op[8];
    char arg1[8];
    char arg2[8];
    char res[8];
} Quad;

Quad code[10] = {
    {"+", "3", "5", "t1"}, // Constant folding → t1 = 8
    {"*", "t1", "x", "t2"}, // Constant propagation → t2 = 8 * x
    {"=", "t2", "_", "y"},
    {"+", "y", "0", "t3"}, // Algebraic identity → t3 = y
    {"=", "t3", "_", "z"}
};

void optimize() {
    printf("--- Optimized Intermediate Code ---\n");
    for (int i = 0; i < 5; i++) {
        // Constant Folding
        if (isdigit(code[i].arg1[0]) && isdigit(code[i].arg2[0])) {
            int val1 = atoi(code[i].arg1);
            int val2 = atoi(code[i].arg2);
            int res = (code[i].op[0] == '+') ? val1 + val2 : val1 * val2;
            printf("FOLD: %s = %d\n", code[i].res, res);
            sprintf(code[i].arg1, "%d", res);
            strcpy(code[i].op, "=");
            strcpy(code[i].arg2, "_");
        }
        printf("%s %s %s %s\n", code[i].op, code[i].arg1, code[i].arg2, code[i].res);
    }
}
```

Comprehensive Viva-Voce Question Bank & Model Answers

Q1. What is the difference between Lex and Yacc?

Lex (Flex) is a lexical analyzer generator converting regular expressions into deterministic finite automata (DFA) to generate token streams ('yylex').

Yacc (Bison) is a syntax parser generator taking context-free grammar specifications to build an LALR(1) parsing table and construct the parse tree ('yparse').

Q2. Why is Left-Recursion forbidden in Top-Down Parsers (LL(1))?

A left-recursive production $A \rightarrow A\alpha \mid \beta$ causes predictive recursive descent parsers to infinitely call function 'A()' without consuming any input tokens from the lookahead buffer, crashing the call stack with stack overflow!

Q3. What are Shift-Reduce and Reduce-Reduce conflicts in Yacc?

- **Shift-Reduce Conflict:** Parser cannot decide whether to shift the next input token or reduce the current stack elements (e.g., Dangling-else ambiguity). Resolved in Yacc by defaulting to Shift or setting '%left'/'%right' precedence.
- **Reduce-Reduce Conflict:** Parser finds two distinct grammar rules whose right-hand sides match the stack top. Resolved by reducing via the rule declared earlier in the grammar.

Q4. What is Backpatching and why is it needed in One-Pass Compilers?

In single-pass compilation, forward jump target line numbers (e.g., target label of 'if (condition) goto ???') are unknown when parsing the condition. **Backpatching** creates lists of pending jump instructions ('makelist'), merges them ('merge'), and fills in the resolved jump address once the destination block is generated ('backpatch').

Q5. Explain the role of 'yywrap()' and 'yyval' in Lex/Yacc communication.

- **'yywrap()':** Called by Lex when 'EOF' is encountered. Returning 1 terminates lexical analysis; returning 0 switches to another input file pointer ('yyin').
- **'yyval':** Global shared union variable used by Lex to pass semantic values (integer constants, token string names, struct pointers) to the Yacc parse stack!